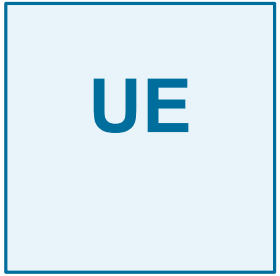
Unreal Engine: Development

Week 1 Notes • Project Introduction & Unreal Engine Review

Instructor: Joss Moo-Young • GAME3023 • Game Programming



combat arena

Week 1

Project Introduction • Unreal Engine Review

- Introduce the term-long Unreal Combat Arena project.
- Clarify how this course is different from a tutorial-based Unreal course.
- Review the Unreal concepts students are expected to reuse.
- Start team formation, project setup, research, and arena planning.
- Prepare for Checkpoint 1: Concept & Development Plan.

Course Identity

A modern take on classic Unreal development.

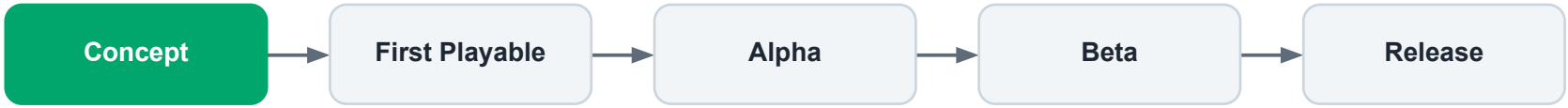
- Students build a focused combat arena using current Unreal Engine tools.
- The project draws on Unreal's roots in arena-based gameplay and level iteration.
- The emphasis is practical development: research, implement, test, revise, and explain.

Mission Statement

- Reconnect students with Unreal Engine’s arena-based development roots through a modern, self-directed project.
- Emphasize creative level design, practical gameplay systems, contemporary engine tools, and independent technical problem solving.
- Use documentation, AI-assisted development, class notes, instructor guidance, and peer feedback as part of a professional workflow.

Semester Roadmap

The same project grows from concept to release.



Week 1 focus: set the project direction and make the project ready to build.

Do not build everything this week. Build a plan strong enough that next week's blockout work has direction.

Part 1

What this course is asking you to do

Where This Course Fits

- Foundations: learned the Unreal editor, Actors, Blueprints, materials, input, collision, audio, UI, and basic C++ use.
- Systems: learned character/animation systems, aiming/firing, AI Controllers, Behavior Trees, EQS, and Perception.
- Development: now combine those skills into one substantial playable Unreal project.
- Capstone: later you will apply this production experience to a larger original project.

Progression Through the Unreal Sequence



Learn the tool | Learn subsystems | Build a game | Extend/optimize | Apply at scale

The Big Shift This Semester

- You will not be shown every feature step-by-step before you need it.
- You will be given project requirements, recommended systems, and documentation paths.
- You are expected to research, experiment, debug, and make technical decisions.
- Joss acts more like a technical lead: short guidance, then project development time.

Typical Weekly Class Pattern

- 20–30 minutes: instructor introduction, review, demo, or project guidance.
- 2.5 hours: supervised development, team work, testing, research, and feedback.
- Outside class: 3–6 hours of continued implementation, documentation, and problem solving.
- Each week should leave the project in a better, more testable state.

Part 2

The Unreal Combat Arena project

Project Overview

- Design and develop one original combat arena in Unreal Engine 5.
- Build the playable space primarily with Unreal's native level/blockout/modeling tools.
- Implement combat, hazards, pickups, feedback, match flow, and opponents.
- Use either AI-controlled opponents or human/networked opponents.
- Finish with a playable release build and a technical presentation.

Individual or Team Project

- You may work individually or in teams of up to three students.
- A larger team should demonstrate greater breadth, complexity, polish, or reliability.
- Every team member must make meaningful technical contributions.
- Everyone should be able to explain the project architecture and their own work.

Two Valid Opponent Paths

Students choose one path. Both are acceptable.

Path A: AI Opponents

- AI Controller
- NavMesh
- Behavior Tree or State Tree
- Perception / sensing
- Attack, damage, death, respawn

Path B: Human Opponents

- Listen server / simple LAN
- Replicated player movement
- Server-authoritative damage
- PlayerState / GameState scoring
- PIE multiplayer testing

What the Project Is Not

- Not a commercial-quality shooter.
- Not an original 3D art portfolio assignment.
- Not a networking infrastructure course with accounts, matchmaking, anti-cheat, or back-end services.
- Not a tutorial recreation where everyone builds the exact same thing.
- Not a reason to add huge features that cannot be completed.

What the Project Should Become

- A readable, navigable arena with intentional spaces and routes.
- A functional combat loop: spawn, move, fight, take damage, die, respawn, score.
- Interactive level mechanics that affect movement or combat.
- A reliable opponent solution: AI bots or human/networked players.
- A stable, packaged build that can be demonstrated and discussed technically.

Part 3

Assessments: checkpoints and milestones

Assessment Pattern

Type	Purpose	Weighting	Mindset
Checkpoint	Show meaningful progress and a realistic plan	Smaller	Are you on track?
Milestone	Demonstrate a required state of functionality	Larger	Does the build meet the stage?
Presentation	Explain the game and the development work behind it	Separate final item	Can you communicate what you built?

Semester Deliverables

Week	Deliverable	Focus	Weight
2	Checkpoint 1	Concept & Development Plan	5%
4	Milestone 1	First Playable	15%
6	Checkpoint 2	Interactive Gameplay Progress	5%
9	Milestone 2	Alpha	15%
11	Checkpoint 3	Feature Integration	5%
13	Milestone 3	Beta	20%
14	Checkpoint 4	Release Candidate Review	5%
15	Milestone 4 + Presentation	Release + Demo/Reflection	20% + 10%

Checkpoint 1 Is Next Week

- A short explanation of the arena concept and intended experience.
- A sketch, diagram, or digital layout of the arena.
- Initial routes, spaces, verticality, spawn areas, and risk/reward ideas.
- A preliminary list of planned mechanics, hazards, pickups, and combat ideas.
- A decision about AI opponents or human/networked opponents.
- Unreal project created and basic team workflow established.

Milestone 1 Preview: First Playable

- Due Week 4.
- The arena blockout should be complete enough to run through and evaluate.
- Player scale, traversal, routes, collision, and spawn planning should be visible.
- Basic materials/lighting should begin to communicate spaces and gameplay meaning.
- Combat is not required yet; the focus is the playable arena foundation.

Important Reminder

A week introduces a focus; it does not mean every related feature must be complete that week.

- Complex systems may span multiple weeks.
- Checkpoints look for meaningful progress and risk awareness.
- Milestones expect a defined functional state.

Part 4

AI-assisted development and research expectations

AI-Assisted Development

- Students are encouraged to responsibly use AI-assisted development tools as part of their workflow for research, debugging, prototyping, documentation, and code or Blueprint assistance.
- AI-generated solutions must be understood, tested, integrated, and appropriately modified by the student.
- Students remain responsible for the functionality, quality, and technical understanding of all submitted work.

What AI Can Help With

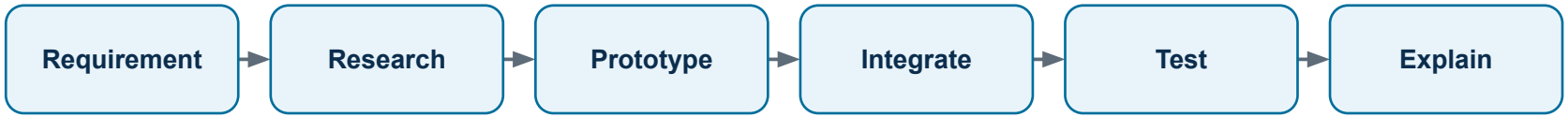
- Explaining Unreal terminology or architecture.
- Suggesting Blueprint or C++ implementation approaches.
- Helping interpret errors or unexpected behavior.
- Generating small prototype snippets or pseudocode.
- Helping create task lists, risk lists, and documentation drafts.
- Finding which Unreal documentation page or system to investigate.

What AI Cannot Replace

- Your technical understanding.
- Your testing and debugging responsibilities.
- Your ability to explain what the project does and how it works.
- Your judgment about scope, performance, maintainability, and suitability.
- Your responsibility to follow the course and college AI-use requirements.

Professional Problem-Solving Loop

Documentation, AI assistance, experimentation, and instructor feedback all support this loop.



Core Unreal Documentation for Week 1

Use official docs first when possible.

Creating a New Project

Use this when creating and configuring your starting project.

Open documentation

Level Editor

Review the editor panels used to construct levels and place Actors.

Open documentation

Unreal Terminology

Useful reference for Actors, Components, Pawns, Controllers, GameMode, GameState, and Brushes.

Open documentation

Source Control

Review Unreal source control workflow and expectations.

Open documentation

Blueprints Visual Scripting

Core reference for visual scripting in the Unreal Editor.

Open documentation

Part 5

Unreal Engine review: concepts you will reuse

Unreal Project Anatomy

- A project is opened through its .uproject file.
- Levels/Maps are stored as .umap assets.
- Gameplay content usually lives inside the Content folder.
- Blueprints, materials, meshes, sounds, and UI assets are all project assets.
- Organized folders make a semester-long project easier to review and grade.

Recommended Folder Structure

A simple structure is enough. Consistency matters more than complexity.

Content / GAME3023

- Maps
- Blueprints
- Materials
- UI
- Audio
- VFX
- Characters
- Docs

Avoid mystery folders and one giant “Stuff” directory.

Actors, Components, and Reusable Gameplay

- Most things placed in a level are Actors.
- Components give Actors useful functionality: mesh, collision, audio, movement, camera, particles, etc.
- Reusable Blueprint Actors are ideal for doors, platforms, jump pads, hazards, pickups, and triggers.
- If you copy-paste logic everywhere, the project becomes hard to debug by Alpha.

Actor Composition Example

A reusable hazard Actor might be built from several components.

BP_DamageHazard Actor

- Static Mesh
- Collision Volume
- Audio / VFX
- Damage Logic

Design goal: configure hazard behaviour without rebuilding the same Blueprint every time.

Blueprint and C++ Expectations

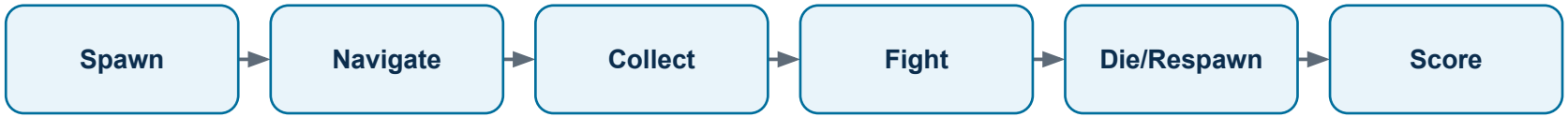
- You may use Blueprints, C++, or a practical combination of both.
- Choose the approach that best supports the system you are building.
- Blueprints are excellent for rapid iteration, level-specific behaviour, UI, and designer-facing logic.
- C++ may be useful for reusable gameplay foundations or systems that benefit from code organization.
- You should be able to explain your choice.

Gameplay Framework Review

- GameMode: rules, match setup, spawn logic; server-only in multiplayer contexts.
- GameState: shared match state such as score, timer, match phase.
- PlayerState: per-player data such as score, kills, deaths, name, team.
- PlayerController: input and player-facing control logic.
- Pawn/Character: the physical avatar that moves through the arena.

Combat Arena Gameplay Loop

The loop should become playable by Alpha, but it starts with planning today.



Part 6

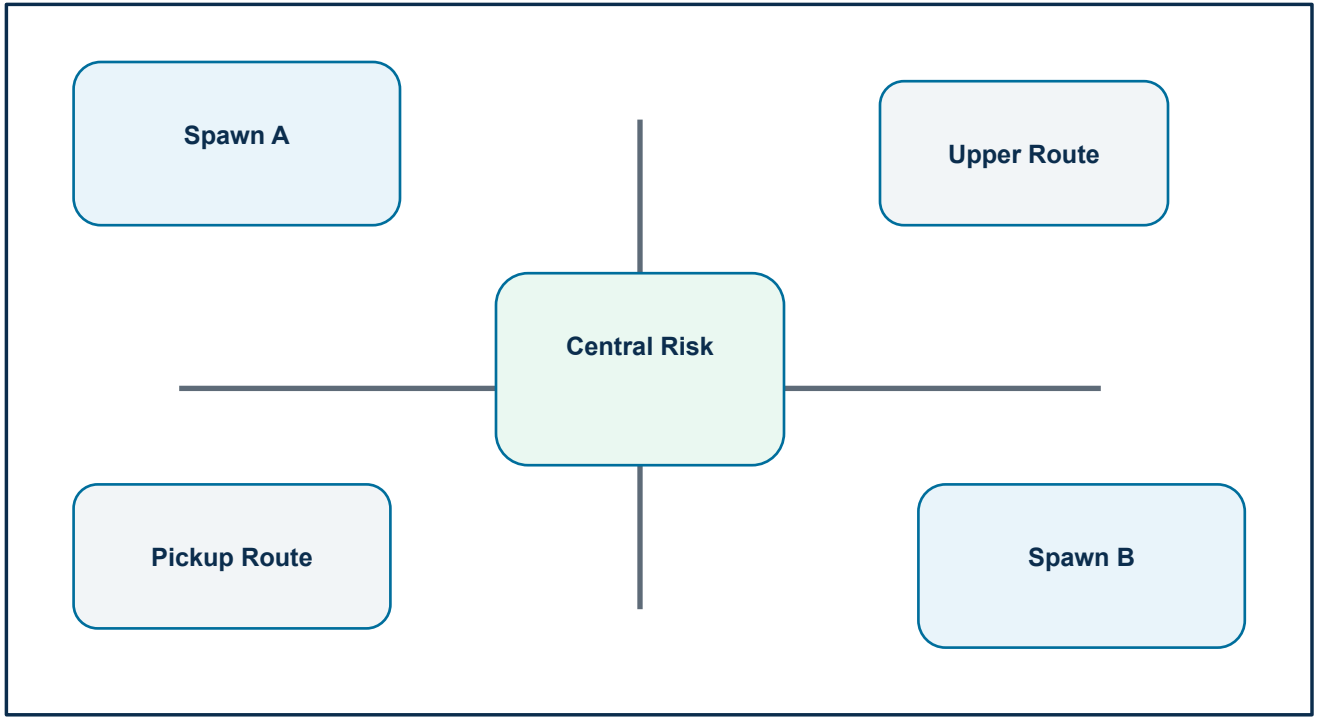
Arena planning: make the space before the systems

Arena Concept Questions

- What is the theme or fantasy of the arena?
- What should the player feel while moving through it?
- Is it close-range, vertical, fast, dangerous, chaotic, strategic, or trap-heavy?
- What makes your arena different from a generic room with enemies?
- What technical features will make the arena interactive?

Example Top-Down Arena Plan

This is not a required layout; it shows the kind of thinking expected in Checkpoint 1.



A useful Week 1 plan identifies spaces, routes, risk/reward, and likely combat flow.

Good Arena Plans Show Intent

- Main routes and alternate routes.
- Possible spawn areas.
- High-risk/high-reward spaces.
- Potential pickup locations.
- Areas where combat is likely to occur.
- Possible vertical movement or shortcuts.
- Where hazards or interactive mechanics might matter.

Scale: Your First Design Constraint

- A combat arena must feel appropriate for the player character speed, camera, and weapon range.
- Spaces that are too large can feel empty and slow.
- Spaces that are too tight can make combat frustrating or unreadable.
- Use the player character early and often to test scale.
- Do not wait until Week 4 to discover the map feels wrong.

Scope Calibration

	Too Small / Missing	Right Scope	Too Large / Risky
Arena	One hallway	One complete arena	A full campaign
Combat	No opponent	One reliable combat style	Ten weapon classes
Environment	Static room	Several meaningful mechanics	Huge physics sandbox
Presentation	Gray only	Readable materials/lighting	Original AAA art set

Routes and Flow

- A strong arena usually offers multiple ways to reach important locations.
- Avoid a single obvious corridor between spawn points.
- Think about how players rotate through the map after spawning.
- Ask: where does a player go after taking damage?
- Ask: where does a player go when chasing an opponent?

Verticality

- Height differences can create interesting tactical choices.
- Upper routes may offer visibility but expose the player.
- Lower routes may provide safety but slow the player down.
- Lifts, ramps, stairs, jump pads, and moving platforms can all support vertical flow.
- Verticality should serve gameplay, not just decoration.

Sightlines and Combat Readability

- Long sightlines support ranged combat and sniping.
- Short sightlines support close-range encounters and ambushes.
- Too many long sightlines can make spawning unfair.
- Too many blind corners can make combat chaotic.
- Use walls, platforms, cover, and elevation to shape what players can see.

Spawn Planning

- Spawn points should not place players directly into unavoidable danger.
- Spawns should avoid giving immediate unfair advantage.
- In multiplayer, spawn fairness becomes especially important.
- In AI projects, bot spawning and player spawning still shape pacing.
- You can revise spawn placement later after testing.

Risk and Reward

- Powerful pickups should usually require the player to make a choice or accept danger.
- High ground can be powerful, so make access and exposure matter.
- Hazards are more interesting when players can intentionally use or avoid them.
- Shortcuts should save time but may increase risk.
- The best arenas create decisions, not just movement paths.

Theme Without Art Overload

- You are not expected to create professional 3D assets from scratch.
- A strong theme can be communicated through shapes, colours, lighting, materials, names, and gameplay mechanics.
- Industrial: elevators, crushers, warning lights, metal/concrete materials.
- Sci-fi: jump pads, teleporters, emissive paths, moving platforms.
- Dungeon: traps, gates, lava, narrow routes, reward rooms.

Part 7

From idea to Unreal project

Starting Project Setup

- Create a new Unreal project using an appropriate game template.
- Use a clear project name such as GAME3023_TeamName_ArenaName.
- Set up a main map/level for the arena.
- Create top-level folders before adding many assets.
- Make sure every team member can open the project successfully.

Source Control / Backup Expectations

- A semester project needs version history or reliable backup practices.
- Commit or back up regularly, especially before major changes.
- Do not rely on one laptop copy as the only project version.
- Large Unreal projects can become difficult to share late in the term.
- Decide the team workflow before the project grows.

Team Workflow

- Agree on who owns the arena plan, gameplay systems, opponent path, UI/feedback, and documentation.
- Avoid having two people edit the same map file at the same time without coordination.
- Keep a simple task list: To Do, In Progress, Testing, Done.
- Record technical problems as they appear instead of trying to remember them later.
- Every member should understand the whole project, not just one isolated piece.

Simple Team Task Board

Use any tool: spreadsheet, Trello, GitHub Projects, Notion, Teams, or a shared document.

To Do	In Progress	Testing	Done
Sketch arena	Create project	Open on team PCs	Team agreement
Research docs	Folder setup	Sketch review	Submit CP1

Development Notes You Should Keep

- Arena sketches and revisions.
- Feature list and cut-list.
- Opponent-path decision: AI or networked human players.
- Technical risks and research links.
- AI/tool usage notes, including what was used and how it was changed/tested.
- Bug list and open questions for Joss.

Part 8

Week 1 work session

Week 1 In-Class Work Plan

The goal is planning and readiness, not implementation completion.

- [ ] Form an individual/team plan and confirm group size.
- [ ] Create the Unreal project and verify it opens correctly.
- [ ] Create a basic folder structure for maps, Blueprints, materials, UI, audio/VFX, and documentation.
- [ ] Discuss arena theme and combat style.
- [ ] Create at least one rough arena sketch or layout draft.
- [ ] Choose initial opponent path: AI-controlled or human/networked.
- [ ] List three environmental mechanics and two hazards you might build later.
- [ ] Record open questions and technical risks for next week.

For Next Week: Checkpoint 1

- Bring an arena concept that can be explained in under one minute.
- Bring a sketch, diagram, or digital plan showing the proposed arena layout.
- Identify major spaces, routes, spawn areas, verticality, pickups, hazards, and interactions.
- Have an Unreal project created with a sensible folder structure.
- Be ready to explain why your scope is achievable.
- Be ready to explain whether you are pursuing AI opponents or networked human opponents.

Checkpoint 1: What Joss Should Be Able to See

- You have a viable project idea.
- You understand the game loop you are aiming for.
- You have a physical arena plan, not just a theme.
- You have thought about scale, routes, verticality, and flow.
- You have identified technical features and risks.
- You are ready to begin blockout next week.

Useful Documentation Links

Bookmark these now. You will use them repeatedly.

Creating a New Project

Project Browser, templates, implementation settings, and project location.

Open documentation

Level Editor

Default editor layout, panels, modes, viewport, Details, Outliner, and Content Browser.

Open documentation

Modeling Mode

Creating and editing 3D geometry directly in Unreal.

Open documentation

Geometry Brush Actors

BSP/brush-style blockout workflows and brush concepts.

Open documentation

Viewport Controls

Navigation and object manipulation in the editor viewport.

Open documentation

Common Week 1 Pitfalls

- Starting with a giant idea before making a playable space.
- Choosing a theme that requires too much original art.
- Ignoring scale until after the blockout is built.
- Not deciding team roles or workflow early enough.
- Planning both AI and networking when only one opponent path is required.
- Letting AI generate ideas/code without understanding or testing the result.

Part 9

End of Week 1

Main Takeaway

Your first job is to make the project buildable.

- A good plan makes Week 2 blockout faster and more useful.
- A controlled scope gives you time to iterate, test, and polish.
- Creativity matters most when it is connected to gameplay.

Before Next Class

- [ ] Project created and shared/backed up.
- [ ] Team roles and communication decided.
- [ ] Arena sketch/digital plan ready.
- [ ] Opponent path selected or narrowed.
- [ ] Initial mechanic/hazard/pickup ideas listed.
- [ ] Technical risks and research links recorded.
- [ ] Questions prepared for Joss.

Week 1 End

- Next week: Arena Concept & Level Planning.
- Checkpoint 1 due: Concept & Development Plan.
- After Checkpoint 1, the project moves into the two-week level blockout phase.
- Come ready to build.